

```

leftover = 1/leftover - next_term
output.append(next_term)
error = abs(get_number(output) - x)
return(output)

```

Listing 5-6: Finding continued fractions from decimal numbers

In this case, we include a `length_tolerance` term just like before. We also add an `error_tolerance` term, which allows us to exit the algorithm if we get an approximation that's "close enough" to the exact answer. To find out whether we are close enough, we take the difference between `x`, the number we are trying to approximate, and the decimal value of the continued fraction terms we have calculated so far. To get that decimal value, we can use the same `get_number()` function we wrote in [Listing 5-5](#).

We can try our new function easily as follows:

```
print(continued_fraction_decimal(1.4142135623730951, 0.00001, 100))
```

We get the following output:

```
[1, 2, 2, 2, 2, 2, 2, 2]
```

We can write this continued fraction as follows (using an approximate equal sign because our continued fraction is an approximation to within a tiny error and we don't have the time to calculate every element of an infinite sequence of terms):

$$1.4142135623730951 \approx 1 + \frac{1}{2 + \frac{1}{2 + \frac{1}{2 + \frac{1}{2 + \frac{1}{2 + \frac{1}{2}}}}}}$$

Notice that there are 2s all along the diagonal in the fraction on the right. We've found the first seven terms of another infinite continued fraction whose infinite expansion consists of all 2s. We could write its continued